

App Dev Project Report

1. Student Details

Name: Mr. Hemant Uppal

Roll Number: 23F2001648

Email: 23f2001648@ds.study.iitm.ac.in

About Me: Aspiring Data Scientist, AI Engineer with strong skills in Python, SQL, Machine Learning, and data analysis. Experienced in building data-driven solutions, business analytics, and web applications using Flask. Passionate about solving real-world problems using AI, analytics, and automation with a strong analytical mindset and hands-on project experience.

2. Project Details

Project Title: Hospital Management System - V2 (Swasth Bharat)



Problem Statement:

The healthcare sector in India faces significant challenges in managing patient appointments, doctor availability, and clinical records efficiently. Existing systems are often paper-based or fragmented, leading to long wait times, miscommunication between departments, and poor patient experience. Swasth Bharat addresses these challenges by providing a unified, role-based web application for hospitals to manage doctors, patients, and appointments seamlessly.

Approach:

The application was built using Flask as the backend framework following an MVC-inspired structure. It implements three distinct user roles: Admin, Doctor, and Patient, each with a dashboard and set of permissions. The backend uses SQLAlchemy, sqlite as the ORM for database interactions, and Jinja2 for server-side HTML rendering. All business logic, authentication, and access control were implemented manually.

3. AI/LLM Declaration

Area of Use	Extent	Details
Frontend UI / Dashboard Design	~40%	Patient, Doctor, and Admin dashboard HTML/CSS generated with AI assistance based on detailed requirements and model specifications provided by the developer
Error Handling Patterns	~10%	AJAX error handling structure and toast notification patterns suggested by AI; integrated and adapted manually
Jinja2 Template Wiring	~15%	Macro definitions and template variable wiring guided by AI; all route-side context variables defined manually
All Backend Logic	0%	All Flask routes, SQLAlchemy queries, authentication, role checks, and business logic written manually by the developer
Database Models	0%	All SQLAlchemy model definitions written manually

Total estimated AI usage: **25–30%**, primarily in frontend HTML/CSS generation and template structure. All final decisions, debugging, integration, and implementation logic were performed by the developer.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database interactions
Jinja2	Server-side template engine for dynamic HTML rendering
Flask-Login	User session management and authentication
Flask-WTF / CSRF	CSRF protection on all AJAX POST requests
Werkzeug	Password hashing (generate_password_hash / check_password_hash)
SQLite	Lightweight relational database
HTML5 / CSS3 / Vanilla JS	Frontend structure, styling, and AJAX interactions
Google Fonts	DM Serif Display, DM Sans, Syne, JetBrains Mono for dashboard typography
Claude (Anthropic AI)	Frontend dashboard UI generation and error handling patterns

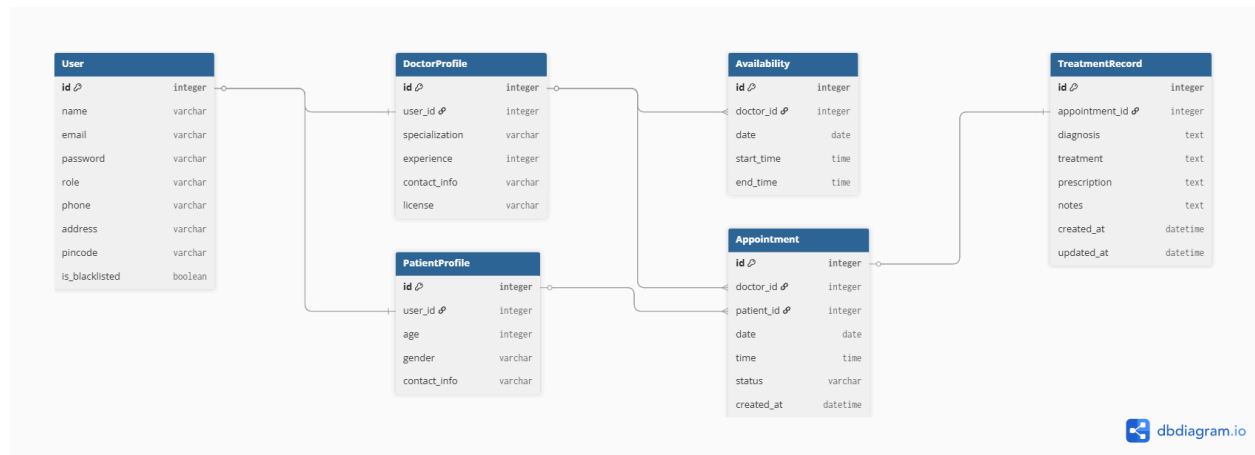
5. Database Schema / ER Diagram

Tables

Table	Key Columns
User	id, name, email, password (hashed), role (admin/doctor/patient), phone, address, pincode, is_blacklisted
DoctorProfile	id, user_id (FK→User), specialization, experience, contact_info, license
PatientProfile	id, user_id (FK→User), age, gender, contact_info
Appointment	id, doctor_id (FK→DoctorProfile), patient_id (FK→PatientProfile), date, time, status (scheduled/completed/cancelled), created_at
Availability	id, doctor_id (FK→DoctorProfile), date, start_time, end_time — UniqueConstraint(doctor_id, date, start_time)
TreatmentRecord	id, appointment_id (FK→Appointment, UNIQUE), diagnosis, treatment, prescription, notes, created_at, updated_at

Relationships

- One-to-One: User → DoctorProfile (each doctor user has one profile)
- One-to-One: User → PatientProfile (each patient user has one profile)
- One-to-Many: DoctorProfile → Appointment (a doctor has many appointments)
- One-to-Many: PatientProfile → Appointment (a patient has many appointments)
- One-to-Many: DoctorProfile → Availability (a doctor has many availability slots)
- One-to-One: Appointment → TreatmentRecord (each appointment has at most one treatment record)



6. API Resource Endpoints

Authentication

Endpoint	Method	Description
/doctorlogin	GET/POST	Doctor login — validates credentials and redirects to doctor dashboard
/patientlogin	GET/POST	Patient login — validates credentials and redirects to patient dashboard
/doctorregister	GET/POST	Doctor self-registration with profile creation
/patientregister	GET/POST	Patient self-registration with profile creation
/logout	GET	Logs out the current user and clears session

Patient Routes

Endpoint	Method	Description
/patient/dashboard	GET	Renders patient dashboard with upcoming/past appointments, doctors, and profile
/patient/book	POST (JSON)	Book an appointment — validates slot availability and creates Appointment record
/patient/appointment/<id>/cancel	POST	Cancel a scheduled appointment by ID
/patient/profile/update	POST (JSON)	Update patient name, contact, age, gender, address
/patient/doctor/<id>/availability	GET (JSON)	Fetch available slots for a doctor for the next 7 days
/patient/doctors	GET (JSON)	List all doctors; supports ?specialization= filter

Doctor Routes

Endpoint	Method	Description
/doctordash	GET	Renders doctor dashboard with stats, appointments, patients, and availability
/doctor/appointment/update	POST (JSON)	Mark appointment as completed or cancelled
/doctor/availability/add	POST (JSON)	Add a new availability slot for a given date and time range
/doctor/availability/delete	POST (JSON)	Remove an existing availability slot by ID
/doctor/patient/<id>/appointments	GET (JSON)	Get all appointments for a specific patient (for modal view)
/doctor/treatment/save	POST (JSON)	Save or update a treatment record for an appointment

Admin Routes

Endpoint	Method	Description
/admindash	GET	Admin dashboard — stats, all doctors, patients, appointments, search panel
/admin/doctor/add	POST (JSON)	Admin creates a new doctor account and DoctorProfile

/admin/doctor/edit	POST (JSON)	Admin updates doctor User and DoctorProfile fields
/admin/patient/edit	POST (JSON)	Admin updates patient User and PatientProfile fields
/admin/user/blacklist	POST (JSON)	Toggle blacklist/unban for a doctor or patient user
/admin/search	GET (JSON)	Server-side search across doctors and patients by name/email/specialization

7. Architecture and Features

Architecture Overview

File / Directory	Purpose
app.py	Main Flask application entry point : registers all routes, initialises db, seeds admin
models.py	SQLAlchemy model definitions (User, DoctorProfile, PatientProfile, Appointment, Availability, TreatmentRecord)
admin_routes.py	All admin-facing routes and the create_admin() seed function
doctor_routes.py	Doctor dashboard route and all doctor AJAX endpoints
patient_routes.py	Patient dashboard route and all patient AJAX endpoints
/templates/	Jinja2 HTML templates (admindash.html, doctordash.html, patient_dashboard.html, login/register pages)
/static/	CSS overrides, JavaScript files, any static assets

Implemented Features

Patient Dashboard:

- Register and log in securely with hashed passwords
- Browse all registered doctors by specialization with emoji-coded department cards
- View a doctor's 7-day availability and book appointments via slot picker modal
- View upcoming appointments with one-click cancellation
- View full health history (past appointments with diagnosis, treatment, prescriptions)
- Edit personal profile (name, age, gender, contact, address) with live ID card preview

Doctor Dashboard:

- View today's and weekly appointment summary with stat cards
- Mark appointments as completed or cancelled in real time
- Manage weekly availability — add and delete time slots via 7-day grid
- View all assigned patients with modal showing full appointment history
- Save treatment records (diagnosis, treatment plan, prescriptions, notes) for appointments
- Treatment History panel with complete records for all patients

Admin Dashboard:

- System overview with stats: total doctors, patients, appointments, completed vs. cancelled counts
- Add new doctor accounts directly (no self-registration required for admin-created doctors)
- Edit doctor and patient profiles (name, email, phone, specialization, experience, etc.)
- View all appointments with status filter tabs (All / Scheduled / Completed / Cancelled)
- Live search across doctors and patients by name, email, specialization, or license
- Blacklist or unban any doctor or patient : blacklisted users cannot log in
- Admin account pre-seeded programmatically on first run — no admin registration page

System-wide:

- Role-based access control : each route checks `current_user.role` before proceeding
- CSRF protection on all POST/AJAX requests via Flask-WTF `csrf_token()`
- Blacklist enforcement at login : blacklisted users are redirected with a flash message
- All AJAX responses return structured JSON { success, message, data } for consistent error handling
- Toast notification system on all dashboards for real-time user feedback

8. Additional Notes

- All passwords are stored as Werkzeug hashed strings
 - Blacklisted users are blocked at the login route level with a clear flash message.
 - The TreatmentRecord table links one-to-one with Appointment via a UNIQUE foreign key, preventing duplicate records per appointment.
 - All AJAX endpoints return JSON and are protected with X-CSRFToken headers.
 - The patient dashboard uses the `spec_emoji` Jinja2 custom filter registered on the Flask app for department icons.
-

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1sHhx0_rbhZo6-hku_a0jicLe1Rzfvltc/view?usp=sharing

(Accessible to all with "View" permission.)
